

12주차 스터디

교재 : 자연어 처리와 컴퓨터비전

분량 : 7장

과제 기간 : ~2025-05-26

07. 트랜스포머

- **Transformer, 신경망 아키텍처**
- 트랜스포머의 주요 기능
 - 기존의 순환 신경망과 같은 순차적 방식이 아닌, **병렬로 입력 시퀀스를 처리하는 기능**
 - 긴 시퀀스의 경우 트랜스포머 모델을 순환 신경망 모델보다 훨씬 빠르고 효율적으로 처리한다
 - 기존의 순차 처리(Sequential Processing)이나 반복 연결(Recurrent Connections)에 의존하지 않는다
 - 입력 토큰 간의 관계를 직접 처리하고 이해할 수 있도록 하는 **셀프 어텐션(Self-attention)**을 기반으로 한다
 - 모델이 재귀나 합성곱 연산 없이 **입력 토큰 간의 관계를 직접 모델링할 수 있다**
- 트랜스포머 모델의 학습
 - 대용량 데이터셋에서 매우 효율적
 - 데이터 양이 많은 기계 번역과 같은 작업에 적합
 - 언어 모델링 및 텍스트 분류 작업에 매우 효과적
 - 광범위한 자연어 처리 작업에서 높은 효율을 보임
 - 장기적인 종속성을 포함하는 작업에서 주로 사용 - ex) 기계 번역, 언어 모델링, 텍스트 요약
 - 자연어 처리 분야에서 널리 사용되고 영향력이 큰 모델
 - 학습 방식 : **오토 인코딩(Auto-Encoding)**, **자기 회귀(Auto Regressive)**

- 트랜스포머 구조

- 오토 인코딩 방식

- 랜덤하게 문장 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업을 수행
 - 예측되는 토큰의 양옆에 있는 토큰들을 참조하기 때문에 양방향 구조를 갖는다
→ Encoder



- 양방향 구조 (그림 A)

- '트랜스포머 모델은 성능이', '다양한 구조로 활용된다'를 입력 받고
 - '높고'를 출력으로 학습하는 양방향 구조
 - 반면 자기 회귀 방식은 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞추는 작업을 수행

- 단방향 구조(그림 B)

- 입력 : '트랜스포머 모델은 성능이'
 - 출력 : '높고'로 학습하는 단방향 구조
 - 예측되는 토큰의 왼쪽에 있는 토큰들만 참조하기 때문에 단방향 구조를 가지며 이를 디코더(Decoder)라고 한다

- 이번 장에서 소개하는 트랜스포머 모델들은 양방향성 구조의 인코더(오토 인코딩) 또는 단방향성 디코더(자기 회귀)를 사용하는 공통적인 특징을 갖고 있다

표 7.1 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

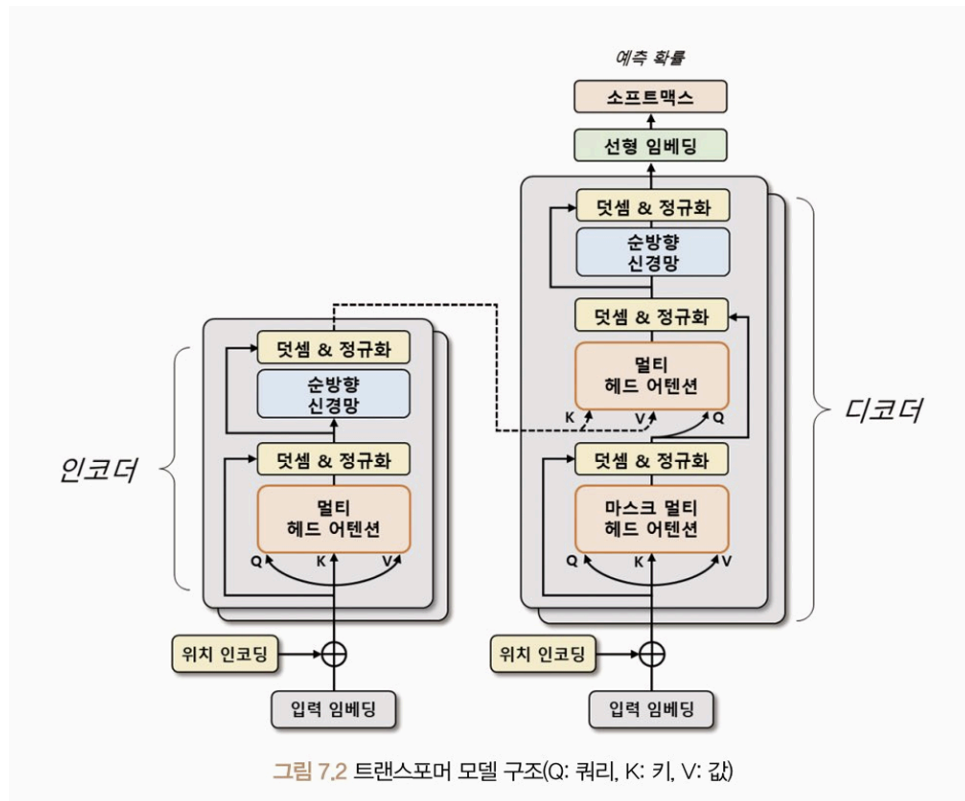
- ELECTRA : 인코딩에 생성적 적대 신경망의 판별기를 활용한 모델
- T5와 BART는 다양한 자연어 처리 작업에 대해 학습을 수행한다

Transformer

- 딥러닝 모델 중 하나, 기계 번역, 챗봇, 음성 인식 등 다양한 자연어 처리 분야에서 많은 성과를 내는 모델
- 순환 신경망이나 합성곱 신경망과 달리 **Attention Mechanism**만을 사용해 시퀀스 임베딩을 표현
- **트랜스포머의 어텐션 메커니즘**
 - 인코더와 디코더의 상호작용
 - 입력 시퀀스의 중요한 부분에 초점을 맞춰 문맥을 이해
 - 적절한 출력을 생성
 - 인코더 : 입력 시퀀스 임베딩, 고차원 벡터로 변환
 - 디코더 : 인코더의 출력을 입력으로 받아 출력 시퀀스를 생성
 - 어텐션 메커니즘 : 인코더와 디코더 단어 사이의 상관관계를 계산 → 중요한 정보에 집중
 - 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지를 파악해 번역이나 요약문 생성과 관련된 작업 등을 수행할 수 있게 된다
- **트랜스포머 모델의 장점**
 - 기존의 순환 신경망 기반 모델보다 학습 속도가 빠르고 병렬 처리가 가능하다
 - 대규모 데이터셋에 높은 성능을 보임

- 임베딩 과정에서 문장 전체의 정보를 고려하므로 문장의 길이가 길어지더라도 성능이 유지된다

• 트랜스포머 모델의 구조



- 트랜스포머의 **인코더와 디코더**는 두 부분으로 구성
- 각각 N개의 **트랜스포머 블록(Transformer Block)**으로 구성
- 이 블록은 **멀티 헤드 어텐션(Multi Head Attention)**과 **순방향 신경망**으로 구성
 - **멀티 헤드 어텐션** : 쿼리, 키, 값 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법
 - 이 과정에서 쿼리와 각 키의 유사도를 계산
 - 해당 유사도를 가중치로 사용해 값 벡터를 합산한다
 - 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체한다
 - 결국 입력 시퀀스의 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신한다
- **순방향 신경망** : 이 과정에서 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용

- 여러 개의 선형 계층으로 구성
 - 앞선 순방향 신경망의 구조와 동일하게 입력 벡터에 가중치를 곱하고 편향을 더하며 활성화 함수를 적용
 - 이 과정에서 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신됨
- 트랜스포머에서는 입력 시퀀스 데이터를 소스(Source)와 타겟(Target) 데이터로 나눠 처리한다
 - ex) 영어를 한글로 번역
 - 생성하는 언어인 한글을 타겟 데이터로 정의, 참조하는 언어인 영어를 소스 데이터로 정의
 - **인코더**는 소스 시퀀스 데이터를 **위치 인코딩(Positional Encoding)**된 입력 임베딩으로 표현해 트랜스포머 블록의 출력 벡터를 생성
 - 이 출력 벡터는 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성된다
 - **디코더**도 유사하게 트랜스포머 블록으로 구성
 - **마스크 멀티 헤드 어텐션(Masked Multi-Head Attention)**을 사용해 타겟 시퀀스 데이터를 순차적으로 생성시킴
 - 이때 디코더 입력 시퀀스들의 관계를 고도화하기 위해 인코더의 출력 벡터 정보를 참조
 - 최종적으로 생성된 디코더 출력 벡터는 선형 임베딩으로 재표현되어 이미지나 자연어 모델에 활용됨
 - 이러한 과정을 통해 트랜스포머는 챗봇, 기계 번역, 음성 인식 등 다양한 자연어 처리 작업에서 좋은 성능을 보임

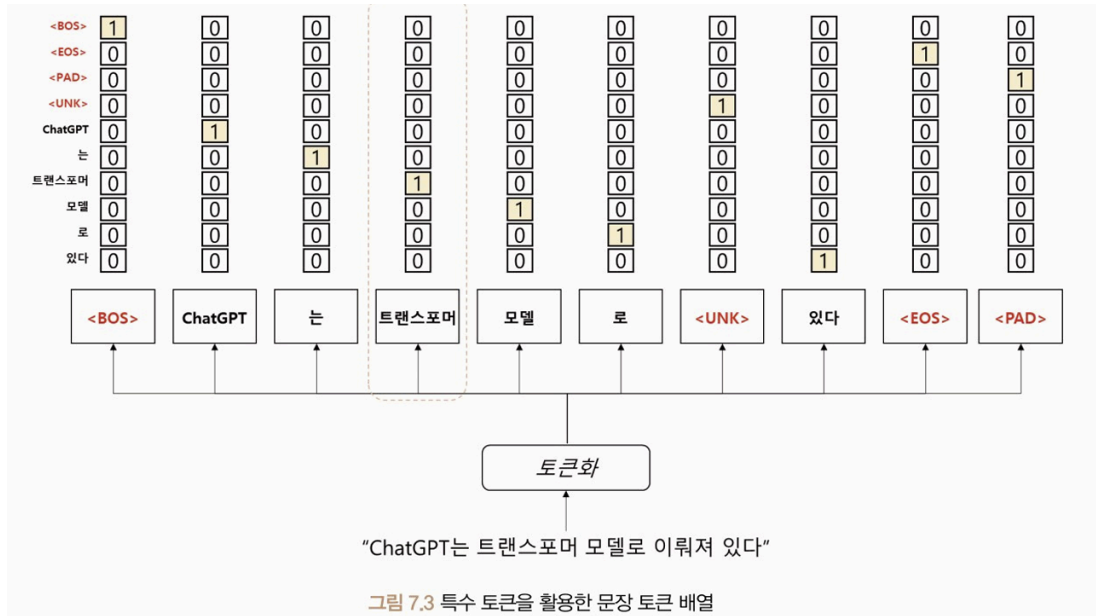
입력 임베딩과 위치 인코딩

- 트랜스포머 모델에서 입력 시퀀스의 각 단어는 임베딩 처리되어 벡터 형태로 변환된다
- 트랜스포머 모델은 순환 신경망과 달리 입력 시퀀스를 병렬 구조로 처리하기 때문에 단어의 순서 정보를 제공하지 않는다

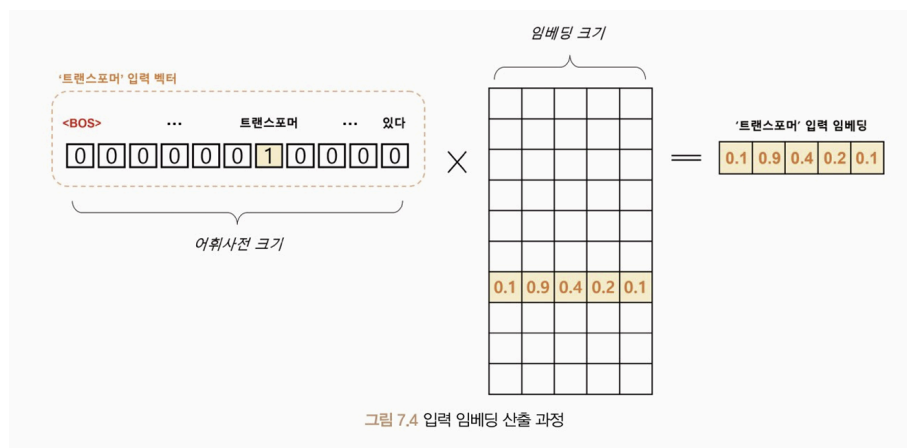
- 그러므로 **위치 정보를 임베딩 벡터에 추가 → 단어의 순서 정보를 모델에 반영해야 한다**
- 위치 인코딩은 입력 시퀀스의 순서 정보를 모델에 전달하는 방법이다
- 각 단어의 위치 정보를 나타내는 벡터를 더해 임베딩 벡터에 위치 정보를 반영한다
- 위치 인코딩 벡터는 $\sin$ 함수와 $\cos$ 함수를 사용해 생성된다
 - 각 토큰의 위치를 각도로 표현해 $\sin$ 함수와 $\cos$ 함수로 위치 인코딩 벡터를 계산
 - 토큰의 위치마다 동일한 임베딩 벡터를 사용하지 X, **각 토큰의 위치 정보를 모델이 학습 가능하다**
- **임베딩 벡터와 위치 정보가 결합된 최종 입력 벡터를 생성**
- 위치 인코딩 벡터를 추가함으로써 모델은 **단어의 순서 정보를 학습**할 수 있게 된다
- 위치 인코딩은 입력 시퀀스의 순서 정보를 보존하기 위한 중요한 방법 중 하나

특수 토큰

- 트랜스포머는 단어 토큰 이외의 **특수 토큰을 활용해 문장을 표현**
 - 입력 시퀀스의 시작과 끝을 나타내거나 마스킹(Masking) 영역으로 사용된다
1. 모델이 입력 시퀀스의 시작과 끝을 인식할 수 있게 해준다
 2. 마스킹을 통해 일부 입력을 무시할 수 있다
 - ex) 번역 모델에서는 디코더의 입력 시퀀스에서 현재 위치 이후의 토큰을 마스크해 이전 토큰만을 참조한다
 - 그림 7.3 : 문장에 특수 토큰을 적용한 문장 토큰 배열



- 특수 토큰, 자연어 처리에서 일반적으로 사용
 - BOS : 문장의 시작
 - EOS : 문장의 종료
 - UNK : 어휘 사전에 없는 단어, 모르는 단어 / 모델이 이전에 본 적이 없는 단어를 처리할 때 사용
 - PAD : 모든 문장을 일정한 길이로 맞추기 위해 사용, 짧은 문장의 빈 공간을 채울 때 사용
- 이렇게 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현한다
 - 그림 7.4 : 문장 토큰 배열 중 '트랜스포머'라는 원-핫 벡터가 입력 임베딩으로 변환되는 과정



- 입력 임베딩으로 변환되는 과정은 Word2Vec 방법과 동일
- 어휘 사전의 크기를 V , 입력 임베딩 차원이 d 일 때, '트랜스포머'의 원-핫 벡터는 $[1, V]$ 크기를 갖는다
- 원-핫 벡터는 임베딩 행렬 $[V, d]$ 에 의해 $[1, d]$ 크기의 벡터로 변환된다
- N 개의 문장이 최대 S 개의 토큰 길이를 가질 때 $[N, S, V]$ 크기의 원-핫 벡터 텐서는 $[N, S, d]$ 크기의 임베딩 텐서로 변환된다
- 이 임베딩 텐서는 입력으로 사용되고 트랜스포머의 모든 계층에서 공유되는 텐서로 사용된다

트랜스포머 인코더

- 트랜스포머 인코더 : 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행
- 각 인코더 계층
 - 멀티 헤드 어텐션과 순방향 신경망으로 구성
 - 입력 데이터에 대한 정보 추출 → 다음 계층으로 전달
 - 인코더 계층에서 위치 정보를 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더해 준다
 - 그 후 산출된 인코더 계층의 출력이 디코더 계층으로 전달된다
 - 그림 7.5 : 트랜스포머 인코더 블록의 연산 과정

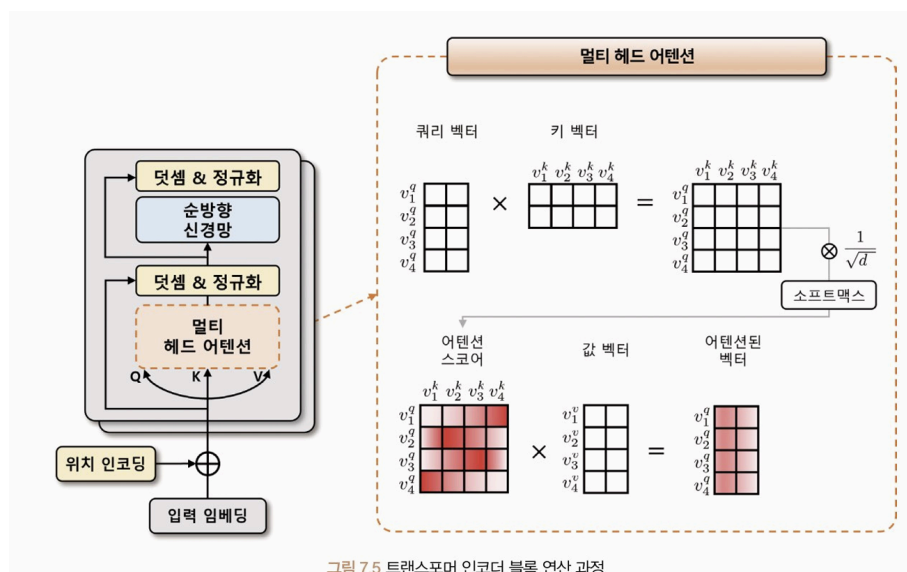


그림 7.5 트랜스포머 인코더 블록 연산 과정

- 위치 인코딩이 적용된 소스 데이터의 입력 임베딩을 입력으로 받는다
 - 입력 임베딩은 선형 변환을 통해 3개의 임베딩 벡터를 생성
 - 각각 쿼리(Q), 키(K), 값(V) 벡터라고 정의한다
- 쿼리 벡터 : 현재 시점에서 참조하고자 하는 정보의 위치를 나타낸 벡터
 - 인코더의 각 시점마다 생성
 - 현재 시점에서 질문이 되는 벡터를 의미한다
 - 이 벡터를 기준으로 다른 시점의 정보를 참조한다
- 키 벡터: 쿼리 벡터와 비교되는 대상
 - 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터
 - 인코더의 각 시점에서 생성
- 값 벡터 : 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할
- 쿼리, 키, 값 벡터는 모델 학습을 통해 의도한 의미의 값을 갖게 된다
- 쿼리와 키 벡터의 연관성은 내적 연산으로 [N, S, S] 어텐션 스코어 맵을 사용한다

수식 7.2 어텐션 스코어 계산식

$$score(v^q, v^k) = \text{softmax} \left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}} \right)$$

- 셀프 어텐션 과정에서의 연산
 - 쿼리, 키, 값 벡터를 내적해 어텐션 스코어를 구한다
 - 이 스코어 값에 벡터 차원의 제곱근만큼($d^{1/2}$) 나눠 보정한다
 - 이 보정값은 벡터 차원이 커질 때 스코어 값이 같이 커지는 문제를 완화하기 위해 적용
 - 보정된 어텐션 스코어를 소프트맥스 함수를 이용해 확률적으로 재표현하고
 - 이를 값 벡터와 내적하여 셀프 어텐션된 벡터를 생성한다

- 이 과정을 반복해 셀프 어텐션된 스코어 맵을 생성한다
- 멀티 헤드(Multi Head)는 이 셀프 어텐션을 여러 번 수행해 여러 개의 헤드를 만든다
 - 각 헤드가 독립적으로 어텐션 수행 후 결과를 합침
 - k개의 셀프 어텐션된 텐서 형태를 구성 $[N, S, d/k]$
 - 이렇게 생성된 k개의 셀프 어텐션 벡터는 임베딩 차원 축으로 다시 병합되어 $[N, S, d]$ 의 형태로 출력된다
- 덧셈, 정규화는 멀티 헤드 어텐션을 통과하기 이전의 입력값 텐서와 출력값 텐서를 더함으로써 학습 시 발생하는 기울기 소실을 완화한다
- 이 값에 임베딩 차원 축을 정규화하는 계층 정규화를 적용
- 순방향 신경망은 두 가지 방법을 적용 - 선형 임베딩과 ReLU로 이뤄진 인공 신경망이나 1차원 합성곱 사용, 이어서 다시 덧셈, 정규화 과정 수행

정리

- 트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성
- 이전 블록의 출력 벡터 → 다음 블록의 입력으로 전달
- 인코더 블록을 통과하며 점차 입력 시퀀스 정보가 추상화된다
- 마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용
- 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터로 활용함
→ 이러한 방식으로 인코더 디코더의 정보 공유가 이뤄짐

트랜스포머 디코더

- 위치 인코딩이 적용된 타겟 데이터의 입력 임베딩을 입력 받는다
- 위치 인코딩은 입력 시퀀스 내에서 각 단어의 상대적인 위치 정보를 전달하는 기법
- 디코더의 입력 임베딩에 위치 정보를 추가 → 디코더가 입력 시퀀스의 순서정보를 학습할 수 있게 됨
- **마스크 멀티 헤드 어텐션**
 - 인코더의 멀티 헤드 어텐션 모듈에 인과성을 반영한 마스크 멀티 헤드 어텐션 모듈로 대체되어 디코더에 적용됨

- 어텐션 스코어 맵 계산 시, 첫 번째 쿼리 벡터가 첫 번째 키 벡터만 바라볼 수 있게 마스크를 씌운다
- 두 번째 쿼리 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크를 씌운다
- 마스크 적용 시 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 된다 →

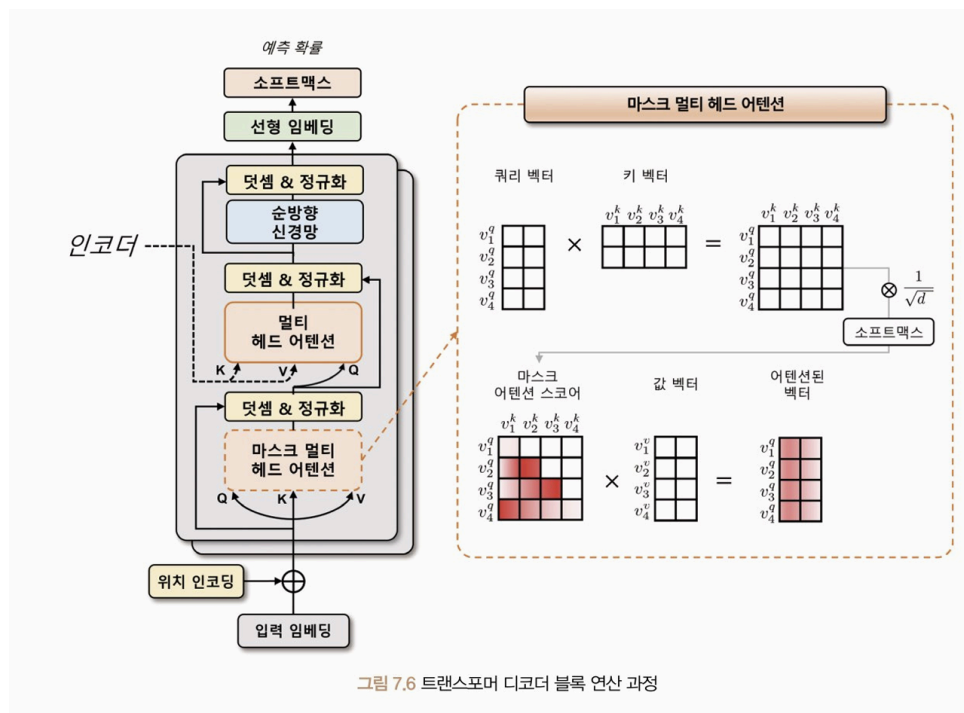
인과성의 보장

- 마스크 영역에 수치적으로 굉장히 작은 값인 $-\text{inf}$ 마스크를 더해줌으로써 해당 영역의 어텐션 스코어 값이 0에 가까워진다
- 소프트맥스 계산 시 해당 영역의 가중치가 0이 되므로,

마스크 영역 이전에 있는 입력 토큰들에 대한 정보를 참조하지 않게 됨

→ 이렇듯 마스크 멀티 헤드 어텐션 모듈은 인코더의 멀티 헤드 어텐션 모듈과 유사하지만, 인과성 보장하면서 셀프 어텐션 수행이 가능

- 트랜스포머 디코더 블록의 연산 과정



- 타깃 데이터가 쿼리 벡터로 사용
- 인코더의 소스 데이터가 키와 값 벡터로 사용
- 따라서 쿼리 벡터는 타깃 데이터의 위치 정보를 포함한 입력 임베딩과 위치 인코딩을 더한 벡터가 된다

- 이후 쿼리, 키, 값 벡터를 이용해 어텐션 스코어 맵 계산, 소프트맥스 함수 적용, 어텐션 가중치 구함
- 최종으로 어텐션 가중치와 값 벡터를 가중합하여 멀티 헤드 어텐션의 출력 벡터를 얻을 수 있음
- 디코더의 멀티 헤드 어텐션은 인코더의 멀티 헤드 어텐션과 거의 동일, 디코더에서는 셀프 어텐션을 방지하기 위해 마스킹이 적용
- 트랜스포머의 디코더도 인코더처럼 여러 개의 트랜스포머 디코더 블록으로 구성
- 이전 트랜스포머 디코더 블록 산출물 → 다음 트랜스포머 디코더 블록의 입력으로 전달
- 디코더는 이전 시점에서의 정보를 현재 시점에서 활용할 수 있게 된다
- 마지막 디코더 블록의 출력 텐서 $[N, S, d]$ 에 선형 변환 및 소프트맥스 함수를 적용해 각 타깃 시퀀스 위치마다 예측 확률을 계산할 수 있게 된다
- 디코더는 타깃 데이터를 추론할 때 토큰 또는 단어를 순차적으로 생성시키는 모델
- 입력 토큰을 순차적으로 나타내는 방법은 빈 값을 의미하는 PAD 토큰을 사용하는 것
- “'ChatGPT'는 트랜스포머 모델로 이뤄져 있다”라는 문장을 추론할 때 디코더의 입출력 예시를 살펴보자

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD], [PAD], [PAD]

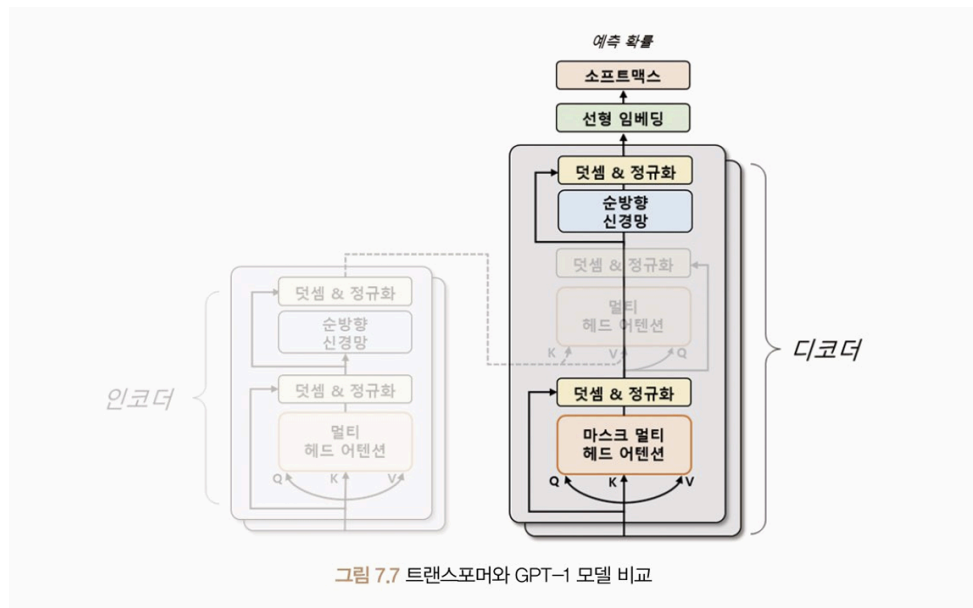
GPT

- GPT, Generative Pre-trained Transformer
 - 트랜스포머 기반 언어 모델
 - ELMo와 같이 대규모 말뭉치로 사전 학습된 모델
 - 다양한 다운스트림 작업에서 우수한 성능을 보인다
- 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델
 - *인코더 : 입력 문장의 특징을 추출하는 데 초점
 - *디코더 : 입력 문장과 출력 문장 간의 관계를 이해하고 출력 문장을 생성하는 데 초점
 - 이러한 특성 때문에 디코더를 쌓아 모델을 구성하는 것이 자연어 생성과 같은 언어 모델 작업에서 더 높은 성능을 발휘한다
- GPT 모델은 대규모 텍스트 데이터셋으로 사전 학습해 초기화되며, 특정 자연어 처리 작업에 맞게 미세 조정해 사용한다
- GPT 모델은 발표순으로 다른 이름으로 알려져 있으며,
 - 거의 동일한 트랜스포머 구조 사용
 - 버전이 갱신될수록 더 많은 트랜스포머 계층과 데이터를 활용해 학습

GPT-1

- **트랜스포머 구조를 바탕으로 한 단방향(Unidirectional) 언어 모델**
- 단방향 언어 모델은 텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측하는 모델
- 이전 단어들을 차례대로 입력하면서 다음 단어를 예측하는 방식으로 동작
- 장점
 - : 모델의 계산량이 줄어들기 때문에 학습 속도가 빠르고 모델의 크기가 작아질 수 있다
- 이전 단어들에 대한 정보로 다음 단어를 예측할 때 트랜스포머 구조를 활용한다
- 트랜스포머 구조 중 **디코더 부분만 사용**, 인코더-디코더 간 어텐션 연산을 제외

- 디코더 부분만 사용하는 방식으로 모델의 매개변수 수를 줄이면서도 높은 성능을 보일 수 있다



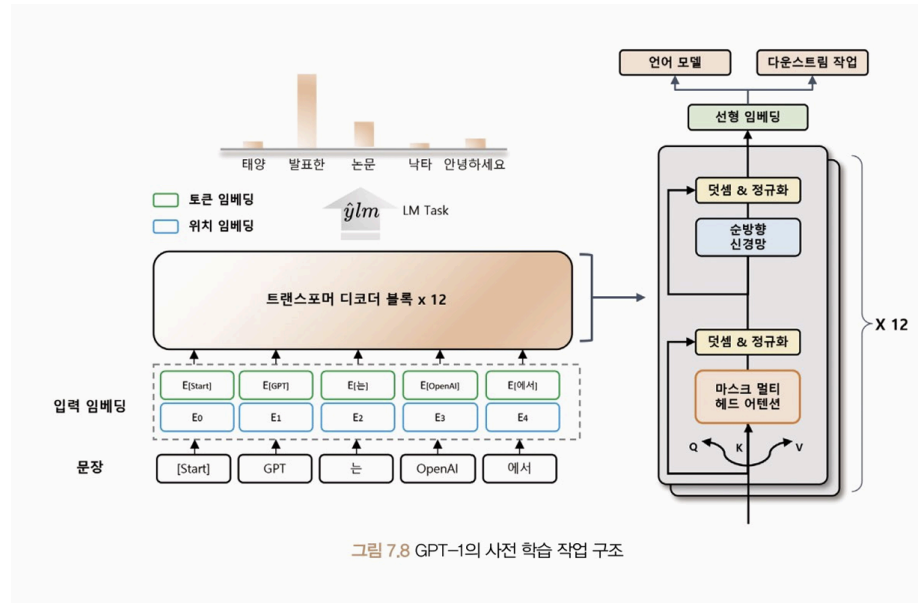
[GPT-1 모델은..]

- 인코더를 사용하지 않고 디코더 부분만 사용해 모델 구성
- 디코더의 두 번째 다중헤드 어텐션은 사용하지 않는다
- 12개의 헤드를 가진 트랜스포머의 디코더를 12층 쌓아 모델을 구성, 약 1억 2천만 개의 매개변수로 학습
- 약 4.5GB의 BookCorpus 데이터셋을 사용해 언어 모델을 사전 학습한다
- 입력 문장을 임의의 위치까지 보여주고 뒤에 이어지는 문장을 모델이 자동으로 예측하게 한다

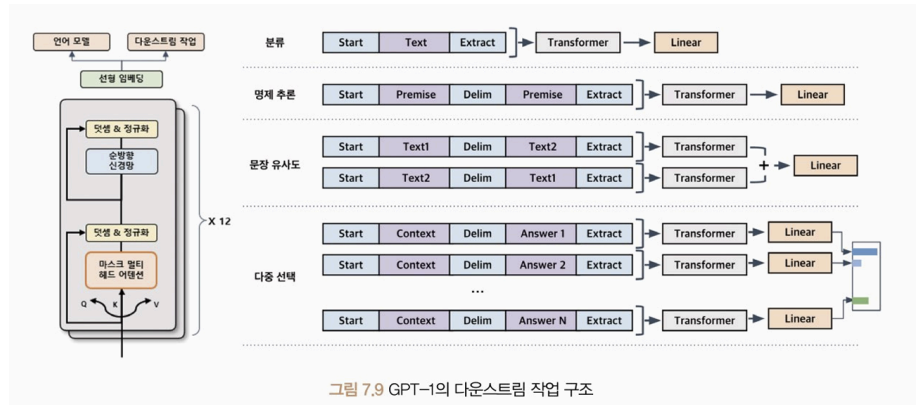
*이 과정에서 별도의 레이블이 필요하지 않기 때문에 비지도 학습으로 학습이 이뤄진다

별도의 레이블이 없는 비지도 학습이므로 컴퓨팅 자원만 충분하다면 제약 없이 학습 가능하다

- GPT-1의 사전 학습 작업 구조



- 입력 문장의 일부만 보고 다음 토큰을 예측하도록 하는 언어 모델을 통해 사전 학습
- 입력 문장을 일정한 길이의 블록으로 나누어 처리
- 각 블록에서는 블록 내 첫 번째 토큰부터 순서대로 다음 토큰을 예측하게 한다
→ 이러한 방식으로 나눠 처리하는 방법은 모델이 장기적인 문맥을 이해할 수 있게 해준다
- 입력 임베딩은 토큰 임베딩과 위치 임베딩을 이용해 계산된다
 - 토큰 임베딩 : 각 토큰을 고정된 차원의 벡터로 매핑
 - 위치 임베딩 : 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터를 매핑
- 미세 조정을 위한 **다운스트림 작업**에서도 GPT-1은 각 작업에 따라 입출력 구조를 바꾸지 X, 언어 모델을 **보조 학습**해 사용할 수 있음
 - 보조 학습, Auxiliary Learning
 - 모델이 주어진 주요 학습 작업 외에도 다른 작업 부가적인 학습 작업을 동시에 수행하는 것
 - 보조 작업에서 학습한 정보를 주요 작업에 전달해 성능을 향상시킬 수 있다



- 다운스트림 작업에서는 원하는 목표에 맞게 모델을 세밀하게 조정해야 한다
 - 일반적인 언어 모델 학습에 사용되는 손실 함수 외에 다운스트림 작업에서 필요한 **추가 손실 함수가 필요하다**
- 두 개의 손실 함수를 사용해 최적화하면
 1. 보조 학습을 통해 언어 모델 개선과
 2. 다운스트림 작업의 목표 달성을 위한 필요 정보 학습이 가능하다
- 다운스트림 작업에서도 특수 토큰이 사용된다
 - : <start>, <delim>, <extract>

GPT-2

- 입력 문장의 길이를 1024 길이까지 처리할 수 있게 됨
- 48개의 디코더 계층을 사용해 더욱 복잡한 패턴을 학습할 수 있게 되었음
- 15억개의 매개변수를 갖게 됨

****GPT-1** : 512 입력 길이, 12개의 디코더 계층, 1200만개의 매개변수

- GPT-2의 모델 구조

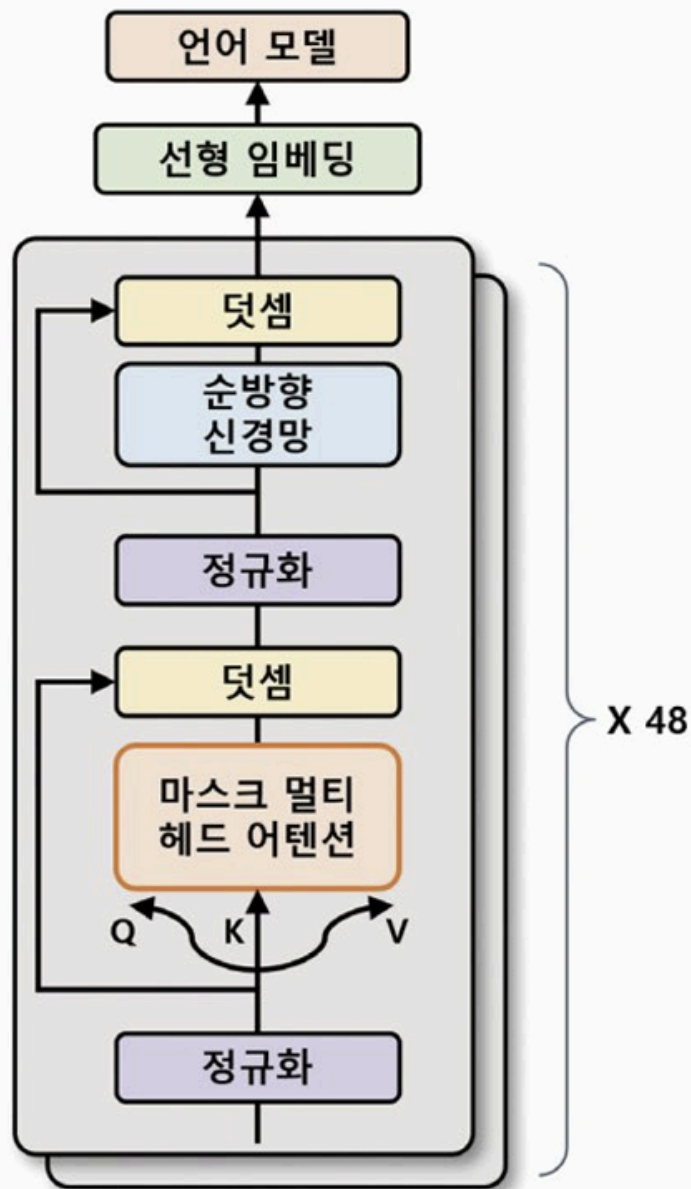
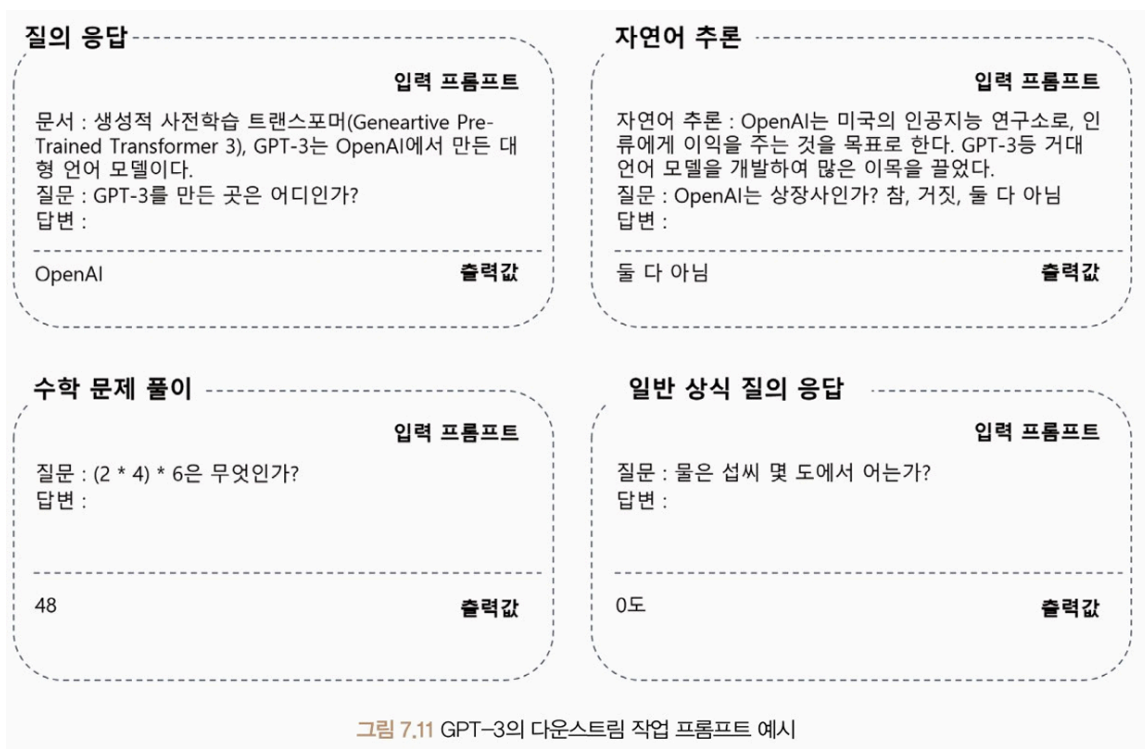


그림 7.10 GPT-2의 모델 구조

- GPT-1보다 훨씬 많은 양의 데이터를 이용해 사전 학습됨(8백만개의 문서, 약 40GB 데이터로 사전 학습 수행)
- 더욱 자연스러운 문장 생성 능력, 더 다양한 분야에서의 활용
- 제로-샷 학습이 가능해졌다
- 별도의 미세 조정 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습 과정에서 특정한 형식의 데이터를 입력한다

GPT-3

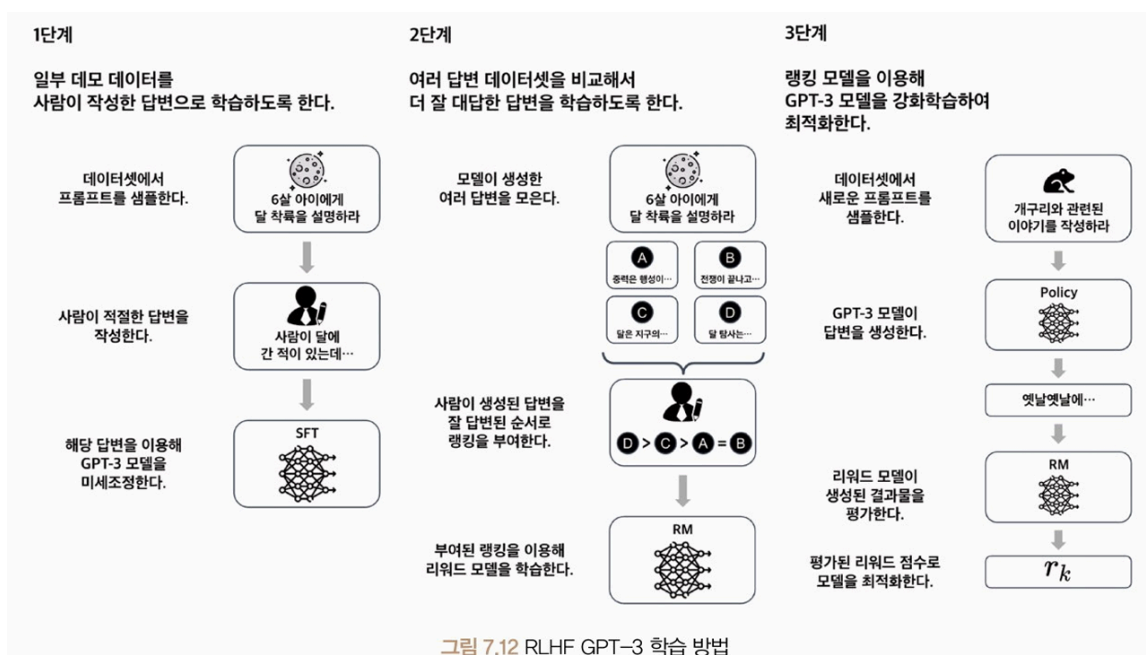
- GPT-2와 거의 동일한 모델 구조
 - 몇몇 모델 매개변수의 변화를 통해 GPT-2보다 약 116배 많은 매개변수를 가진 모델
 - 매우 규모가 큰 모델, 96개의 헤드를 가진 멀티 헤드 어텐션 사용
 - 트랜스포머 디코더 96개의 층
 - 멀티 헤드 어텐션 과정에서 연산을 줄이기 위해 희소 어텐션(Sparse Attention)과 일반 어텐션을 섞어서 사용
 - 토큰 임베딩 크기도 12,888로 대폭 증가
 - 입력의 길이도 2048로 증가, 더욱 긴 문장에 대한 처리 능력이 향상
 - 웹 크롤링, 위키 백과, 서적 등에서 수집한 약 45TB 데이터세트를 이용해 모델을 학습
- *다운스트림 작업으로 학습하진 않았지만 질의응답, 번역, 요약, 문서 생성 등 다양한 다운스트림 작업에서도 높은 성능을 보임
- GPT -3도 GPT-2처럼 프롬프트 형식으로 작업 수행
- GPT-3의 다운스트림 작업 수행 :



- 새로운 도메인에서의 작업에 대해서도 높은 일반화 능력을 보임
 - 질의응답, 수학 문제 풀이, 자연어 추론 등 기존 자연어 처리 분야에서는 이전 기술적 한계를 극복한 성과를 보여줌
- 단점 :
 - 하지만 큰 모델인 만큼 비용이 많이 들고 느림
 - 사전 학습된 데이터에 기반해 작동하기 때문에 데이터가 편향돼 있거나 정확하지 않은 경우 오류가 발생한다
 - 자연어 처리에 대한 일반화 능력이 높지만 인간에 비한 이해력, 상식적 추론 능력이 부족
 - 모델이 생성한 텍스트가 항상 사실이 아니며 편견, 혐오 표현 등이 포함될 수도 있다

GPT-3.5

- InstructGPT, GPT-3의 모델 구조를 그대로 따르면서 새로운 학습 방법인 RLHF를 도입
- 매개변수 수를 줄이고 모델의 자연스러움을 높임
- RLHF의 학습 방법 :



- 데이터세트에서 임의의 프롬프트를 갖고온다
- 이 프롬프트에 대해 사람이 직접 적절한 답을 작성하게 한다
- 이 때 작성된 답변은 모델의 생성 문장과 함께 평가
 - 모델 생성 문장이 자연스러우면 보상, 그렇지 않으면 보상 X
 - 보상을 바탕으로 강화 학습을 통해 모델을 학습

⇒ 이러한 과정을 **지도 미세 조정(Supervised Fine-Tuning, STF)**라고 한다

- RLHF를 통해 학습된 모델은 GPT-3보다 더 자연스러운 문장을 생성함

****하지만 학습 데이터양을 고려하면 전체 프롬프트에 사람이 답변을 다는 것은 현실적으로 불가능**

- 그러므로 GPT 모델이 얼마나 잘 답했는지 평가하는 **보상 모델**을 학습
 - **보상 모델(Reward Model)**로 사람이 평가하고 그에 따른 랭킹을 부여한다
 - 사람이 부여한 랭킹은 생성된 답변의 우수성을 평가한 결과, 이를 보상 모델의 입력으로 사용
 - 보상 모델은 이렇게 생성된 답변과 랭킹 정보를 활용해 학습하며 이를 통해 모델은 더 자연스러운 문장을 생성할 수 있게 된다
- GPT 모델은 시드에 따라 다른 답을 생성하므로 내용이 서로 다른 여러 개의 답변이 생성될 수 있다
- RLHF에서는 주어진 프롬프트(State)에 대해 GPT-3 언어 모델(Policy)이 답변을 생성(Action)하고 이를 이용해 사람 피드백을 기반으로 한 학습된 보상 모델을 평가(Reward)한다
- 이 과정에서 생성된 답변과 랭킹 정보를 활용해 리워드 모델이 학습
- 이를 통해 더 자연스러운 문장 생성이 가능해지는 GPT-3.5가 학습됨
- 이 과정 반복을 통해 더욱 정교한 답변 생성 모델을 학습한다

GPT-4

- 이전 모델과 다르게, 텍스트 데이터뿐만 아니라 이미지 데이터 인식 가능한 **멀티모달(Multimodal)** 모델이다

- GPT-3.5의 입력값 8배, 32,768개의 토큰을 입력으로 받을 수 있다, 25,000개의 단어 처리 가능
- 모델의 매개변수 수나 학습 데이터, 학습 방법이 공개되지는 않았다
- 이전 모델에 비해 높은 성능 향상을 보였지만 언어 모델이 일부 입력에 대해서는 현실적이지 않은 결과를 생성하는 **환각(Hallucination) 현상**을 보임
 - 환각 현상 : 모델이 인간과 같은 추론, 판단 능력을 갖추지 못해서 부적절하거나 오류가 많은 답변을 제시하는 현상

*GPT-4도 트랜스포머 구조, RLHF 기반 학습 했을 것이다